Bevezetés a Rust programozásba

Gembela Gergely

4. óra: Ownership, memóriakezelés, és unsafe

https://github.com/bmeaut/VIAUBXAV090

Ismétlés: heap vs stack allokáció

- A stack
  - Limitált méretű foglalások, *gyorsan*
  - Előre ismert méretű adatok tárolására célszerű használni
- A heap
  - Kezelése legtöbbször manuális, vagy Garbage Collectort igényel
  - Nagyobb/dinamikus foglalások, *nagyobb áron*
  - Az olvasási költség is pointer feloldást igényel

Scope-ok

- Alapvetően egy `{}` közti terület

```
{
  let valami = Box::new(5u64); //valami heap allokál egy u64-et
} // ami itt felszabadításra kerül
```

- Az értékekhez kötött élettartamokkal egy érték garantált elérhetősége módosítható

Ownership (birtoklás)

- A nyelv legfontosabb/egyedibb jellemzője
- Szabályrendszer, amivel garantálható a memóriabiztonság, Garbage Collector (GC) nélkül
- A szabályok fordításkor ellenőrizhetők
  - *Nincs* futásidejű ára (runtime borrow checkek esetében van…)
  - Hozzászokást igényel, sok esetben „akadályozza” a fordítást

Az ownership szabályai

- Minden értéknek pontosan egy tulajdonosa van
- Ha az érték kifut a scope-ból (tehát nem lenne tulajdonosa), megszűnik létezni
  - Amíg ez nem történik meg, **garantáltan** érvényes
  - Ez nem csak stack allokált memóriára vonatkozik

A borrow checker

- A Rust fordító fordításidejű bizonyító komponense
  - Garantálja a memóriabiztonságot
  - Meg lehet kerülni
  - Több esetben nem fogad el logikus kódot
- **Választanunk kell az alábbiak közül:**
  - **Pontosan egy írható referenciánk van**
  - **Bármennyi olvasható referenciánk van**

Miért jó ez nekünk?

- C/C++/egyéb manuálisan memóriakezelt nyelv esetén:
  - Ilyesmivel mindenképpen számolnunk kellett
  - Segít, mert bizonyítja, hogy helyesen dolgoztunk
- GC nyelvekkel szemben:
  - Nincs szükség szüneteltetésre (GC Pause)
  - Determinisztikus
  - Potenciálisan kisebb overhead

Memóriakezelés

- Nem foglalunk/kezelünk kézzel memóriát
  - Ha nagyon szeretnénk, C...
- Különben a modern C++-ból ismerthez hasonló okos pointerekkel kezeljük a heap allokációkat
  - Ezek pedig az ownership rendszer részévé válnak

Lifetimes (élettartamok)

- Minden referenciához tartozik egy *lifetime*
  - Ez határozza meg, hogy meddig *él* (tehát érvényes) egy referencia
  - Egy *lifetime* túlél egy másikat, ha a scope-ja (modernebb Rustban a felhasználási területe) hosszabb annál
  - A leghosszabb lifetime a `'static` (ezek a fordított kódban élő adatokra mutatnak)
  - Kicsit torz analógiával elképzelhetők halmaz-részhalmaz relációként

Lifetimes (élettartamok)

- Lényegében annotációk, amiket a fordító kényszerek betartására és elemzésére használ
  - Fordítási időben a típusok részévé/tulajdonságává válik
  - Futási időben nincs szerepük (nem léteznek futás közben/befolyásolják a generált kódot)
- NLL (Non-Lexical Lifetimes) óta sokkal kevésbé szigorú (már nem blokkoktól, hanem felhasználástól függenek)

Resource Acquisition Is Initialization

- A változók az ownership „owner” szereplői
- A konstruktor jellemzően egy `new` nevű függvény
- A destruktor a `Drop` trait `drop(&mut self)` függvénye

```
struct ToDrop;

impl Drop for ToDrop {
    fn drop(&mut self) {
        println!("dropped!");
    }
}
```

RustViz

- Eszköz a borrow checker logikájának megjelenítésére
- Playground
- Ha nem sikerülne az első feladat megoldása, érdemes kipróbálni

Raw pointerek

- A referenciákhoz hasonlóan `*const T` és `*mut T`
  - Képzés és a dereferálás a C-ben megszokott operátorokkal
- Safe kódból is létrehozhatunk pointereket
  - Ennek ellenére nem jellemző, hogy tisztán Rust kódbázisban rutinszerűen használnánk
  - Dereferálni nem tudjuk ezeket

Unsafe

- Az unsafe Rust sem enged meg bármit, sőt
- Amit megtehetünk:
  - Raw pointer dereferálása
  - Másik unsafe függvény meghívása
  - `mut` statikus változók használata
  - Unsafe trait implementálása
  - Union field-jeinek olvasása

Unsafe

- Továbbra sem:
  - kapcsolja ki a borrow checkert
  - kapcsol ki bármilyen safety checket
- Nem jelenti azt, hogy kifejezetten veszélyes amit csinálunk
- Lehetséges unsafe kódot megfelelő absztrakción keresztül safe kódból használni

Unsafe - mikor jó

- Interface hardver felé
- Interface más nyelvű könyvtárakkal
- Optimalizáció ami máshogy nem lenne lehetséges
- Union használata szándékosan más típusokkal

W4F1: Hibajavítás

- A ZH-ban is várható ilyen jellegű feladat
- A GitHub oldalon mellékelt kódrészlet nem fordítható le, mivel hibákat tartalmaz
  - A cél a kód fordíthatóvá tétele, az eredeti funkcionalitás megőrzésével/javításával

Köszönöm a figyelmet!